Day 4 Notes: Python File Handling & JSON

AI-Resilient Technical Learning Roadmap | First Principles & Systems Practice

1. First Principles: Memory (RAM) vs. Storage (Disk)

RAM (Random Access Memory): Volatile memory. When a Python script creates data like *expenses = [{'item': 'Coffee', 'cost': 4.50}]*, it lives in RAM. The instant the program finishes or closes, RAM is freed and data vanishes.
Disk (SSD / Hard Drive): Non-volatile storage. To preserve data across program runs, we must write data from RAM out to a file on disk.
Plain Text on Disk: Files on disk are raw text sequences, not live Python objects. Python must **serialize** RAM objects into plain text to save them, and **deserialize** plain text back into objects when reading.

2. Operating System Pipelines & The 'with' Context Manager

File Mode	Behavior on open()	Missing File Behavior	Risk Factor
"r" (Read)	Opens existing file for reading	Raises FileNotFoundError	Cannot write or create
"w" (Write)	Opens file for writing; ERASES (truncates) content	Creates new file automatically	Data lost if opened before saving

> **The 'with' Context Manager (Safeguard):**
> *with open('data.json', 'w') as f:* opens a file handle pipeline **f**. The moment execution leaves the indented block, Python automatically flushes buffers and closes the file handle safely, even if a runtime exception occurs inside the block.

3. JSON Translation (json.dump vs. json.load)

json.dump(obj, fp): RAM → Disk. Converts a Python object (*obj*) into JSON text and writes it into an open file handle (*fp*).
json.load(fp): Disk → RAM. Reads raw JSON text from file handle (*fp*) and parses it into matching Python objects (lists, dicts, floats).

```
# Saving to Disk (RAM -> Disk)
with open('expenses.json', 'w') as f:
    json.dump(expenses, f)

# Loading from Disk (Disk -> RAM)
with open('expenses.json', 'r') as f:
    data = json.load(f)  # Reconstructs list of dicts in RAM
```

4. Application Patterns: load_expenses() & save_expenses()

To build resilient applications that handle first-time runs and prevent data loss:

```
import json

def load_expenses():
    try:
        with open('expenses.json', 'r') as f:
            return json.load(f)
    except FileNotFoundError:
        return []  # First-run fallback: returns empty list in RAM

def save_expenses(expenses):
    with open('expenses.json', 'w') as f:
        json.dump(expenses, f)  # Creates file if missing & saves RAM list
```

5. Critical Pitfalls & System Misconceptions

- **Argument Swapping in json.dump:** Writing *json.dump(f, data)* crashes with *AttributeError: 'list' object has no attribute 'write'*. First arg is data, second arg is file handle.
- **Nested Indexing:** If *data = [{'amount': 4.50}]*, calling *data['amount']* raises *TypeError: list indices must be integers or slices, not str*. You must index position 0 first: *data[0]['amount']*.
- **Exception Case Sensitivity:** Writing *except filenotfounderror:* causes a *NameError* because Python standard exception classes use PascalCase (*FileNotFoundError*).
- **Data Loss on Unexpected Crashes:** Calling *expenses.append()* only modifies RAM. If power cuts before *save_expenses()* is called, changes are lost. Always invoke *save_expenses()* immediately after appending!

6. JSON Serialization Rules

JSON is a language-agnostic text standard supporting 6 basic types: **string, number, boolean, null, array, object**. Basic Python types (*list, dict, str, int, float, bool, None*) map cleanly into JSON. Complex objects containing live OS/runtime state (like open file handles, functions, or custom class instances) **cannot** be serialized by *json.dump()* and trigger a *TypeError*.

> **Key Systems Rule:** RAM is for fast execution; Disk is for persistence. Commit RAM changes to disk immediately after user input to ensure zero data loss.